

Microsoft Fabric Banking Analytics Project

End-to-End Banking Analytics Solution using Microsoft Fabric

Overview

The Microsoft Fabric Banking Analytics Project demonstrates the complete lifecycle of building a modern enterprise analytics solution using Microsoft Fabric. The project follows the Medallion Architecture (Bronze → Silver → Gold), enabling data ingestion, transformation, modeling, governance, reporting, collaboration, and version control within a unified analytics platform.

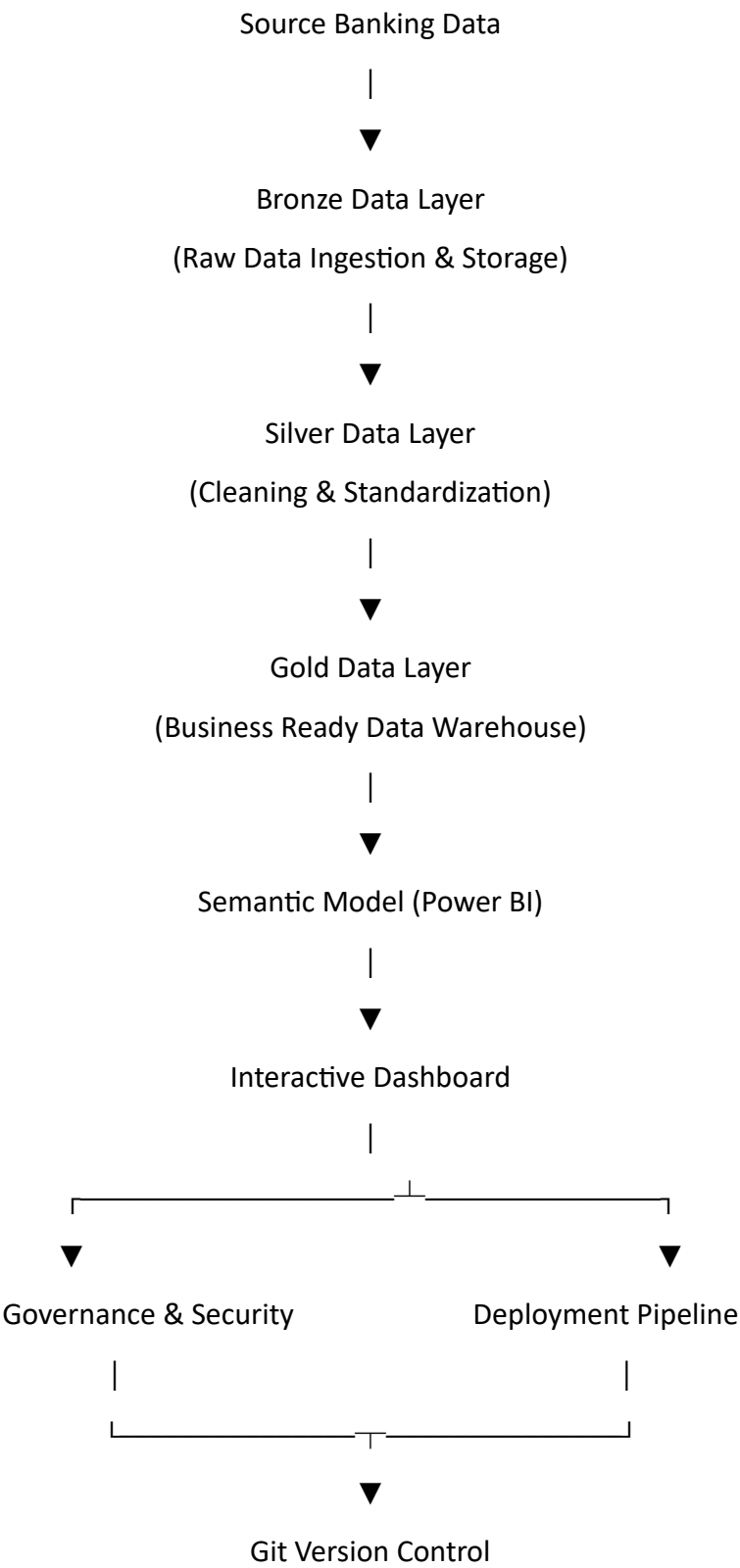
The solution begins with ingesting raw banking datasets into the Bronze layer, progressively transforms and enriches the data through the Silver layer, and finally creates business-ready dimensional models in the Gold layer for analytical reporting. The project concludes with governance, security, deployment, optimization using Materialized Lake Views, and collaborative development through Git Integration.

Project Objectives

The project demonstrates how to:

- Build an end-to-end analytics solution using Microsoft Fabric
 - Implement Medallion Architecture
 - Automate data ingestion using Pipelines
 - Transform data using Dataflow Gen2
 - Store curated data as Delta Tables
 - Create a Star Schema for reporting
 - Build a Power BI Dashboard
 - Implement Governance and Security
 - Configure Deployment Pipelines
 - Optimize queries using Materialized Lake Views
 - Enable collaborative development through Git Integration
-

Solution Architecture:



Module 1: Bronze Data Layer

Overview

The Bronze layer is the first stage of the Medallion Architecture. It is responsible for ingesting raw banking datasets into Microsoft Fabric while preserving the original data without applying business transformations. The data is stored as Delta tables within a Lakehouse, providing a reliable foundation for downstream processing.

The Bronze layer implementation consists of:

- Create Lakehouse
 - Upload Source Files
 - Create Bronze Data Ingestion Pipeline
 - Create Bronze Dataflow Gen2
 - Create Bronze Delta Tables
 - Validate Bronze Layer
-

1.1 Create the Bronze Lakehouse

OBJECTIVE

Create a Microsoft Fabric Lakehouse to centrally store raw banking datasets and Delta tables used throughout the Medallion Architecture.

NAVIGATION

Microsoft Fabric

→ Workspaces

→ fabric_banking_analytics_ws

→ New Item

→ Lakehouse

METHOD

Follow the steps below in sequence:

1. Open the **fabric_banking_analytics_ws** workspace.
2. Click **New Item**.
3. Search for **Lakehouse**.
4. Select **Lakehouse**.
5. Enter the following name:

fabric_banking_medallion_lakehouse

6. Click **Create**.
7. Wait until the Lakehouse is successfully provisioned.
8. Open the Lakehouse.
9. Verify that the following folders are available:
 - Tables
 - Files

EXPECTED RESULT

A Lakehouse named **fabric_banking_medallion_lakehouse** is successfully created.

1.2 Upload the Banking Source Files

OBJECTIVE

Upload the raw banking CSV datasets into the Lakehouse so they can be consumed by Pipelines and Dataflows.

NAVIGATION

fabric_banking_medallion_lakehouse

→ Files

→ Upload

→ Files

METHOD

Follow the steps below in sequence:

1. Open **fabric_banking_medallion_lakehouse**.
 2. Select **Files**.
 3. Click **Upload**.
 4. Select **Files**.
 5. Browse to the banking dataset location.
 6. Upload the following files:
 - users.csv
 - cards.csv
 - transactions.csv
 7. Wait for all files to finish uploading.
 8. Verify that all three CSV files appear under **Files**.
-

1.3 Create the Bronze Data Ingestion Pipeline

OBJECTIVE

Create a Fabric Pipeline that automates the ingestion of raw banking files into the Bronze layer.

NAVIGATION

fabric_banking_analytics_ws

→ New Item

→ Data Pipeline

METHOD

Follow the steps below in sequence:

1. Click **New Item**.
2. Select **Data Pipeline**.
3. Rename the pipeline:

pl_bronze_ingestion

4. Click **Create**.
 5. Add a **Copy Data** activity to the canvas.
 6. Configure the Source connection.
 7. Browse to the uploaded CSV files in the Lakehouse.
 8. Configure the Destination as the Lakehouse.
 9. Enable automatic table creation.
 10. Save the pipeline.
 11. Click **Run**.
 12. Wait for the execution to complete successfully.
-

1.4 Create the Bronze Dataflow Gen2

OBJECTIVE

Create a Dataflow Gen2 that reads the raw Bronze tables, performs minimal ingestion processing, and writes the data back as managed Bronze Delta tables.

NAVIGATION

fabric_banking_analytics_ws

→ New Item

→ Dataflow Gen2

METHOD

Follow the steps below in sequence:

1. Click **New Item**.
2. Select **Dataflow Gen2**.
3. Rename the Dataflow:

df_bronze_standardization

4. Click **Create**.
5. In the Power Query editor, click **Get Data**.
6. Select **Lakehouse** as the source.
7. Choose the following Lakehouse:

fabric_banking_medallion_lakehouse

8. Select the following source files/tables:
 - users
 - cards
 - transactions
9. Load all three queries into the Dataflow.

10. Rename the queries as:

bronze_users

bronze_cards

bronze_transactions

11. Review each query and verify that no business transformations are applied.

12. Configure the Data Destination for each query.

Destination:

Lakehouse

Lakehouse:

fabric_banking_medallion_lakehouse

13. Enable **Replace existing table** (if re-running).

14. Save the Dataflow.

15. Click **Publish**.

16. Wait until the Dataflow refresh completes successfully.

EXPECTED RESULT

The Bronze Dataflow successfully loads all three raw datasets into managed Delta tables without applying business transformations.

1.5 Create Bronze Delta Tables

OBJECTIVE

Persist the raw banking datasets as Delta tables within the Lakehouse to provide scalable and ACID-compliant storage.

NAVIGATION

fabric_banking_medallion_lakehouse

→ Tables

METHOD

After the Bronze Dataflow completes successfully:

1. Open the Lakehouse.
2. Select **Tables**.
3. Verify that the following Delta tables have been created:

bronze_users

bronze_cards

bronze_transactions

4. Open each table.
 5. Preview the data.
 6. Verify that all records have been loaded successfully.
-

EXPECTED RESULT

The Bronze Lakehouse contains the following Delta tables:

- bronze_users
- bronze_cards
- bronze_transactions

1.6 Validate the Bronze Layer

OBJECTIVE

Verify that the Bronze ingestion process has successfully loaded the raw banking data into Delta tables.

NAVIGATION

fabric_banking_medallion_lakehouse

→ SQL Analytics Endpoint

→ New SQL Query

METHOD

Follow the steps below in sequence:

1. Open the SQL Analytics Endpoint.
2. Create a new SQL query.
3. **Validate the Users table:**

```
SELECT COUNT(*) AS TotalUsers  
FROM bronze_users;
```

4. **Validate the Cards table:**

```
SELECT COUNT(*) AS TotalCards  
FROM bronze_cards;
```

5. **Validate the Transactions table:**

```
SELECT COUNT(*) AS TotalTransactions  
FROM bronze_transactions;
```

6. Compare the record counts with the original source files.

7. Verify that no unexpected NULL values or missing records exist due to ingestion issues.

EXPECTED RESULT

All Bronze Delta tables contain the expected number of records, matching the uploaded source datasets and confirming successful data ingestion.

Module 2: Silver Data Layer

Overview

The Silver layer transforms the raw Bronze data into clean, standardized, and business-ready datasets. During this stage, inconsistencies are corrected, missing values are handled, data types are standardized, and related datasets are merged to prepare the data for dimensional modeling in the Gold layer.

Unlike the Bronze layer, the Silver layer focuses on improving data quality while preserving business meaning.

The Silver layer implementation consists of:

- Create Silver Processing Pipeline
- Create Silver Dataflow Gen2
- Rename Bronze Queries
- Data Cleansing
- Data Standardization
- Merge Queries
- Create Silver Delta Tables
- Validate Silver Layer

2.1 Create the Silver Processing Pipeline

OBJECTIVE

Create a Microsoft Fabric Pipeline to orchestrate the execution of the Silver Dataflow and automate the transformation of Bronze data into curated Silver datasets.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ New Item

→ Data Pipeline

METHOD

Follow the steps below in sequence:

1. Open the **fabric_banking_analytics_ws** workspace.
2. Click **New Item**.
3. Select **Data Pipeline**.
4. Rename the pipeline:

pl_silver_processing

5. Click **Create**.
6. Drag a **Dataflow** activity onto the pipeline canvas.
7. Select the Silver Dataflow (to be created in the next exercise).
8. Save the pipeline.
9. Do not execute the pipeline until the Silver Dataflow has been configured.

EXPECTED RESULT

A Silver Processing Pipeline is created and ready to orchestrate the Silver Dataflow execution.

2.2 Create the Silver Dataflow Gen2

OBJECTIVE

Create a Dataflow Gen2 that consumes the Bronze Delta tables and applies cleansing, standardization, validation, and business transformations.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ New Item

→ Dataflow Gen2

METHOD

Follow the steps below in sequence:

1. Click **New Item**.
2. Select **Dataflow Gen2**.
3. Rename the Dataflow:

df_silver_processing

4. Click **Create**.
5. Click **Get Data**.
6. Select **Lakehouse**.
7. Choose:

fabric_banking_medallion_lakehouse

8. Import the following Bronze Delta tables:
 - bronze_users
 - bronze_cards
 - bronze_transactions
9. Load the queries into the Power Query editor.

2.3 Rename Bronze Queries to Staging Queries

OBJECTIVE

Rename the imported Bronze queries to staging queries to distinguish raw source data from transformed Silver datasets.

NAVIGATION

Power Query Editor

→ Queries Pane

METHOD

Follow the steps below in sequence:

Rename the queries as follows:

Existing Query	Rename To
bronze_users	stg_users
bronze_cards	stg_cards
bronze_transactions	stg_transactions

Verify that all three staging queries are visible in the Queries pane.

EXPECTED RESULT

The staging queries are successfully renamed and ready for transformation.

2.4 Transform the Customer Dataset

OBJECTIVE

Clean and standardize customer information to improve data quality before analytical modeling.

NAVIGATION

Power Query

→ stg_users

METHOD

Follow the steps below in sequence:

1. Open **stg_users**.
2. Capitalize the values in the **Gender** column.
3. Remove the \$ symbol from:
 - Income
 - Debt
4. Convert **Income** to Decimal Number.
5. Convert **Debt** to Decimal Number.
6. Verify that all numeric columns have the correct data type.
7. Add a Custom Column named:

Age_Validation_Flag

8. Use the following validation logic:
 - TRUE → Age $\geq$ 18
 - FALSE → Age < 18
9. Rename the transformed query:

silver_customers

2.5 Transform the Card Dataset

OBJECTIVE

Prepare the card dataset by retaining only business-relevant information and ensuring consistent data types.

NAVIGATION

Power Query

→ stg_cards

METHOD

Follow the steps below in sequence:

1. Open **stg_cards**.
2. Review all imported columns.
3. Remove unnecessary columns, if applicable.
4. Verify the data types for:
 - Card ID
 - Customer ID
 - Credit Limit
 - Card Brand
 - Card Type
5. Rename the query:

silver_cards

EXPECTED RESULT

The card dataset is standardized and optimized for downstream modeling.

2.6 Create the Customer-Card Merge

OBJECTIVE

Combine customer and card information into a single curated dataset that simplifies downstream dimensional modeling.

NAVIGATION

Power Query

→ Home

→ Merge Queries

METHOD

Follow the steps below in sequence:

1. Select the **silver_customers** query.
2. Click **Merge Queries**.
3. Choose the **silver_cards** query.
4. Match the tables using:

customer_id

5. Select the **Left Outer Join**.
6. Expand the merged table.
7. Select the required card columns.
8. Rename the merged query:

silver_customer_cards

9. Verify that each customer contains the associated card information.
-

EXPECTED RESULT

A consolidated customer-card dataset is successfully created.

2.7 Configure Silver Delta Tables

OBJECTIVE

Configure the destination Lakehouse and publish the transformed Silver datasets as Delta tables.

NAVIGATION

Power Query

→ Data Destination

METHOD

Follow the steps below in sequence:

For each transformed query:

- silver_customers
- silver_cards
- silver_customer_cards
- silver_transactions

Configure the following destination:

Destination

Lakehouse

Lakehouse - fabric_banking_medallion_lakehouse

Enable:

- Replace existing table
 - Save the Dataflow.
 - Click **Publish**.
 - Wait until the refresh completes successfully.
-

2.8 Execute the Silver Pipeline

OBJECTIVE

Run the Silver Processing Pipeline to automate the execution of the Silver Dataflow.

NAVIGATION

fabric_banking_analytics_ws

→ pl_silver_processing

METHOD

Follow the steps below in sequence:

1. Open **pl_silver_processing**.
 2. Click **Run**.
 3. Monitor the pipeline execution.
 4. Wait until the execution completes successfully.
-

EXPECTED RESULT

The Silver Pipeline executes successfully and refreshes all Silver Delta tables.

2.9 Validate the Silver Layer

OBJECTIVE

Validate the transformed Silver datasets and verify that the expected business transformations have been applied successfully.

NAVIGATION

fabric_banking_medallion_lakehouse

→ SQL Analytics Endpoint

→ New SQL Query

METHOD

Follow the steps below in sequence:

Validate each Silver table.

Validate Customers

```
SELECT COUNT(*) AS TotalCustomers  
FROM silver_customers;
```

Validate Cards

```
SELECT COUNT(*) AS TotalCards  
FROM silver_cards;
```

Validate Customer Cards

```
SELECT COUNT(*) AS TotalCustomerCards  
FROM silver_customer_cards;
```

Validate Transactions

```
SELECT COUNT(*) AS TotalTransactions
```

```
FROM silver_transactions;
```

Review the results and verify:

- Customer data is standardized.
- Currency fields are numeric.
- Gender values are consistently formatted.
- Customer-card relationships are successfully merged.
- Record counts align with the transformation expectations.

EXPECTED RESULT

The Silver layer contains clean, validated, and standardized Delta tables that are ready for dimensional modeling in the Gold layer.

Transformation	Purpose	Business Benefit
Capitalize Gender	Standardize text values	Ensures consistent reporting and filtering
Remove \$ from Income and Debt	Convert currency text to numeric values	Enables calculations and aggregations
Convert Data Types	Ensure correct numeric and date formats	Prevents calculation errors
Age Validation Flag	Identify customers below 18 years	Supports business rule validation
Merge Customer & Card Data	Combine related entities	Simplifies downstream dimensional modeling

Module 3: Gold Data Layer

Overview

The Gold layer is the final stage of the Medallion Architecture. It transforms the cleansed Silver datasets into a business-ready analytical model using dimensional modeling principles. This layer consists of Dimension tables, Fact tables, a Semantic Model, and DAX measures that enable interactive reporting and business intelligence through Power BI.

Unlike the Bronze and Silver layers, the Gold layer is specifically optimized for analytical workloads and reporting performance.

The Gold layer implementation consists of:

- Create Gold Processing Pipeline
 - Create Gold Modeling Dataflow Gen2
 - Import Silver Delta Tables
 - Create Dimension Tables
 - Create Fact Table
 - Configure Gold Delta Tables
 - Create Semantic Model
 - Configure Star Schema
 - Create DAX Measures
 - Validate Gold Layer
-

3.1 Create the Gold Processing Pipeline

OBJECTIVE

Create a Microsoft Fabric Pipeline to orchestrate the execution of the Gold Dataflow and automate the creation of business-ready analytical tables.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ New Item

→ Data Pipeline

METHOD

Follow the steps below in sequence:

1. Open the **fabric_banking_analytics_ws** workspace.
2. Click **New Item**.
3. Select **Data Pipeline**.
4. Rename the pipeline:

pl_gold_processing

5. Click **Create**.
 6. Drag a **Dataflow** activity onto the pipeline canvas.
 7. Configure the activity to execute the Gold Modeling Dataflow.
 8. Save the pipeline.
 9. Do not execute the pipeline until the Gold Dataflow has been configured.
-

EXPECTED RESULT

The Gold Processing Pipeline is successfully created and ready to orchestrate the Gold Dataflow.

3.2 Create the Gold Modeling Dataflow Gen2

OBJECTIVE

Create a Dataflow Gen2 that consumes the Silver Delta tables and prepares business-ready Dimension and Fact tables.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ New Item

→ Dataflow Gen2

METHOD

Follow the steps below in sequence:

1. Click **New Item**.
2. Select **Dataflow Gen2**.
3. Rename the Dataflow:

df_gold_modeling

4. Click **Create**.
5. Click **Get Data**.
6. Select **Lakehouse**.
7. Choose the following Lakehouse:

fabric_banking_medallion_lakehouse

8. Import the following Silver Delta tables:
 - silver_customers
 - silver_cards
 - silver_customer_cards

- silver_transactions

9. Load all queries into the Power Query editor.

EXPECTED RESULT

The Gold Modeling Dataflow successfully imports all Silver Delta tables.

3.3 Create the Customer Dimension Table

OBJECTIVE

Create the Customer Dimension table containing descriptive customer attributes used for filtering, grouping, and reporting.

NAVIGATION

Power Query

→ silver_customers

METHOD

Follow the steps below in sequence:

1. Select the **silver_customers** query.
2. Remove any columns not required for analytical reporting.
3. Verify the customer attributes such as:
 - Customer Key
 - Customer Name
 - Gender
 - Age
 - Credit Score
 - Income
 - Debt
 - Number of Cards
4. Rename the query:

dim_customer

5. Verify that the Customer Key uniquely identifies each customer.
-

3.4 Create the Card Dimension Table

OBJECTIVE

Create the Card Dimension table containing card-related attributes for reporting and analysis.

NAVIGATION

Power Query

→ silver_cards

METHOD

Follow the steps below in sequence:

1. Open the **silver_cards** query.
2. Retain only the business-required columns.
3. Verify the following fields:
 - Card Key
 - Customer Key
 - Card Brand
 - Card Type
 - Credit Limit
 - Card Number
 - Expiration Date
4. Rename the query:

dim_card

EXPECTED RESULT

The Card Dimension table is successfully created.

3.5 Create the Transaction Fact Table

OBJECTIVE

Create the central Fact table containing all banking transactions for analytical reporting.

NAVIGATION

Power Query

→ silver_transactions

METHOD

Follow the steps below in sequence:

1. Open the **silver_transactions** query.
2. Verify that the following fields are available:
 - Transaction Key
 - Customer Key
 - Card Key
 - Transaction Amount
 - Transaction Date
 - Merchant
 - Transaction Type
 - Merchant State
 - Merchant City
3. Remove unnecessary columns, if applicable.
4. Rename the query:

fact_transactions

3.6 Configure Gold Delta Tables

OBJECTIVE

Publish the Dimension and Fact tables as Gold Delta tables within the Lakehouse.

NAVIGATION

Power Query

→ Data Destination

METHOD

Configure the destination for the following queries:

- dim_customer
- dim_card
- fact_transactions

Destination:

Lakehouse

Lakehouse:

fabric_banking_medallion_lakehouse

Enable:

Replace existing table

Click **Publish**.

Wait until the refresh completes successfully.

EXPECTED RESULT

The Gold Delta tables are successfully published into the Lakehouse.

3.7 Execute the Gold Processing Pipeline

OBJECTIVE

Run the Gold Processing Pipeline to refresh all Gold Delta tables.

NAVIGATION

fabric_banking_analytics_ws

→ pl_gold_processing

METHOD

Follow the steps below in sequence:

1. Open **pl_gold_processing**.
 2. Click **Run**.
 3. Monitor the pipeline execution.
 4. Wait for the execution to complete successfully.
-

EXPECTED RESULT

The Gold Processing Pipeline refreshes all analytical tables successfully.

3.8 Create the Semantic Model

OBJECTIVE

Create a centralized Semantic Model that exposes the Gold Layer tables for analytical reporting in Power BI.

NAVIGATION

fabric_banking_medallion_lakehouse

→ New Semantic Model

METHOD

Follow the steps below in sequence:

1. Open the Lakehouse.
2. Select the Gold Delta tables.
3. Click **New Semantic Model**.
4. Name the Semantic Model:

sm_gold_analytics

5. Click **Create**.
 6. Wait for the Semantic Model to be generated.
-

EXPECTED RESULT

A Semantic Model named **sm_gold_analytics** is successfully created.

3.9 Configure the Star Schema

OBJECTIVE

Establish relationships between the Dimension and Fact tables using a Star Schema.

NAVIGATION

Semantic Model

→ Model View

METHOD

Follow the steps below in sequence:

1. Open the Semantic Model.
2. Navigate to **Model View**.
3. Create the following relationships:

From	To
dim_customer.customer_key	fact_transactions.customer_key
dim_card.card_key	fact_transactions.card_key

4. Configure each relationship as:
 - Cardinality = One-to-Many
 - Cross Filter Direction = Single
 - Active Relationship = Enabled
 5. Save the model.
-

EXPECTED RESULT

The Gold model follows a Star Schema with properly configured relationships.

3.10 Create Business DAX Measures

OBJECTIVE

Create reusable DAX measures that provide key business metrics for reporting and dashboarding.

NAVIGATION

Semantic Model

→ Model View

→ New Measure

METHOD

Create the following DAX measures:

- Total Sales
- Total Transactions
- Total Customers
- Total Quantity Sold
- Average Transaction Value
- Average Unit Price

Verify that each measure returns the expected result.

Save the Semantic Model.

EXPECTED RESULT

The Semantic Model contains reusable business KPIs ready for Power BI reporting.

3.11 Validate the Gold Layer

OBJECTIVE

Validate the Gold analytical model and ensure all Dimension tables, Fact tables, relationships, and DAX measures are functioning correctly.

NAVIGATION

fabric_banking_medallion_lakehouse

→ SQL Analytics Endpoint

→ New SQL Query

METHOD

Validate each Gold table.

Validate Customer Dimension

```
SELECT COUNT(*) AS TotalCustomers  
FROM dim_customer;
```

Validate Card Dimension

```
SELECT COUNT(*) AS TotalCards  
FROM dim_card;
```

Validate Transaction Fact

```
SELECT COUNT(*) AS TotalTransactions  
FROM fact_transactions;
```

Verify:

- Dimension tables contain unique business entities.
- Fact table contains transactional records.
- Relationships are correctly configured.
- DAX measures return accurate business values.
- Record counts align with the Silver layer.

EXPECTED RESULT

The Gold Layer is successfully validated and provides a business-ready analytical model that supports Power BI dashboards and enterprise reporting.

Gold Layer Transformation Summary

Component	Purpose	Business Benefit
Customer Dimension	Stores descriptive customer attributes	Enables customer segmentation and filtering
Card Dimension	Stores card-related details	Supports card-based analysis and reporting
Transaction Fact	Stores business transaction records	Provides the foundation for KPI calculations
Semantic Model	Centralizes business logic	Enables self-service analytics and consistent reporting
Star Schema	Optimizes relationships	Improves query performance and simplifies report development
DAX Measures	Encapsulates business calculations	Delivers reusable KPIs across reports and dashboards

Module 4: Governance & Security

Overview

Governance and Security are critical components of every enterprise data platform. Microsoft Fabric provides centralized capabilities to manage workspace permissions, monitor data processing activities, control report publishing, and ensure secure collaboration across teams.

In this module, governance is implemented by assigning workspace permissions, monitoring pipeline and dataflow executions, connecting Power BI Desktop to the Fabric Semantic Model, building and publishing an interactive dashboard, and validating the reporting environment.

The Governance & Security implementation consists of:

- Manage Workspace Access
 - Monitor Pipeline Executions
 - Monitor Dataflow Refresh Activities
 - Connect Power BI Desktop to Fabric
 - Create the Banking Analytics Dashboard
 - Publish the Dashboard to Fabric
 - Validate the Published Report
-

4.1 Manage Workspace Access

OBJECTIVE

Assign appropriate workspace permissions to collaborators, enabling secure access to Microsoft Fabric artifacts while maintaining centralized governance.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ Manage access

METHOD

Follow the steps below in sequence:

1. Open the **fabric_banking_analytics_ws** workspace.
2. Click **Manage access** from the upper-right corner.
3. Select **Add people or groups**.
4. Enter the Microsoft account or organizational email address of the collaborator.
5. Select the appropriate Workspace Role.

Choose one of the following roles:

- Admin
 - Member
 - Contributor
 - Viewer
6. For this implementation, assign the user the following role:

Member

7. Click **Add**.
 8. Verify that the user appears in the workspace access list.
-

4.2 Monitor Pipeline Executions

OBJECTIVE

Monitor pipeline execution history to verify successful data orchestration and identify any failed pipeline runs.

NAVIGATION

Microsoft Fabric

→ Monitor

→ Activities

METHOD

Follow the steps below in sequence:

1. Open **Monitor** from the left navigation pane.
2. Select **Activities**.
3. Use the search box and search for:

pl_

4. Review the execution history of the following pipelines:
 - pl_bronze_ingestion
 - pl_silver_processing
 - pl_gold_processing
5. Verify the following information for each pipeline:
 - Pipeline Name
 - Status
 - Start Time
 - Submitted By
 - Workspace

6. Confirm that each pipeline completed successfully.
7. If required, click a pipeline execution to view additional execution details.

EXPECTED RESULT

All pipeline executions display a **Succeeded** status with successful execution history.

4.3 Monitor Dataflow Refresh Activities

OBJECTIVE

Monitor Dataflow Gen2 refresh operations to ensure successful execution of Bronze, Silver, and Gold data transformations.

NAVIGATION

Microsoft Fabric

→ Monitor

→ Activities

→ Filter

→ Item Type = Dataflow Gen2

METHOD

Follow the steps below in sequence:

1. Open **Monitor**.
2. Select **Activities**.
3. Open the **Filter** menu.
4. Configure the filter as follows:

Item Type

Dataflow Gen2

5. Review the execution history for:
 - df_bronze_standardization
 - df_silver_processing
 - df_gold_modeling
6. Verify:
 - Refresh Status
 - Start Time

- Duration
- Submitted By

7. Confirm that every Dataflow refresh completed successfully.

EXPECTED RESULT

All Dataflow refreshes complete successfully and display a **Succeeded** status.

4.4 Connect Power BI Desktop to Microsoft Fabric

OBJECTIVE

Connect Power BI Desktop to the published Microsoft Fabric Semantic Model to enable report development using centralized business data.

NAVIGATION

Power BI Desktop

→ Get Data

→ Power BI Semantic Models

METHOD

Follow the steps below in sequence:

1. Open **Power BI Desktop**.
2. Click **Get Data**.
3. Select **Power BI Semantic Models**.
4. Sign in using the same organizational account used for Microsoft Fabric.
5. Locate the following Semantic Model:

sm_gold_analytics

6. Select the Semantic Model.
 7. Click **Connect**.
 8. Wait for the model to load.
 9. Verify that all Gold Layer tables and DAX measures appear in the Fields pane.
-

EXPECTED RESULT

Power BI Desktop successfully connects to the Fabric Semantic Model.

4.5 Create the Banking Analytics Dashboard

OBJECTIVE

Create an interactive Power BI dashboard that provides business insights using the Gold Layer Semantic Model.

NAVIGATION

Power BI Desktop

→ Report View

METHOD

Follow the steps below in sequence:

1. Create KPI Cards displaying:
 - Total Sales
 - Total Transactions
 - Total Customers
 - Total Quantity Sold
 - Average Transaction Value
 - Average Unit Price
2. Add a Line Chart showing:

Total Sales by Month

3. Add a Donut Chart displaying:

Total Sales by Transaction Type

4. Add a Treemap showing:

Total Sales by Supplier

5. Add a Column Chart displaying:

Total Sales by Year

6. Add report slicers for business analysis.

7. Apply report formatting including:

- Dashboard Title
- Visual Alignment
- Consistent Theme
- Labels
- Data Colors

8. Save the report.

EXPECTED RESULT

A fully interactive Banking Analytics dashboard is successfully created.

4.6 Publish the Dashboard to Microsoft Fabric

OBJECTIVE

Publish the completed Power BI report to Microsoft Fabric for centralized reporting and collaboration.

NAVIGATION

Power BI Desktop

→ Home

→ Publish

METHOD

Follow the steps below in sequence:

1. Open the completed Banking Analytics report.
2. Click **Publish**.
3. Select the following workspace:

fabric_banking_analytics_ws

4. Click **Select**.
 5. Wait until publishing completes successfully.
 6. Open Microsoft Fabric.
 7. Navigate to the workspace.
 8. Verify that the following artifacts have been created:
 - Global Trust Bank Dashboard (Semantic Model)
 - Global Trust Bank Dashboard (Report)
-

EXPECTED RESULT

The dashboard and associated Semantic Model are successfully published to the Fabric workspace.

4.7 Validate Published Report

OBJECTIVE

Verify that the published report is accessible, interactive, and correctly connected to the underlying Semantic Model.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ Global Trust Bank Dashboard

METHOD

Follow the steps below in sequence:

1. Open the published report.
 2. Verify that all report pages load successfully.
 3. Interact with the dashboard by:
 - Selecting visuals
 - Applying filters
 - Using slicers
 4. Confirm that all visuals update dynamically.
 5. Verify that KPI values match the expected DAX calculations.
 6. Ensure there are no dataset refresh errors.
-

EXPECTED RESULT

The published dashboard functions correctly and accurately reflects the Gold Layer analytical model.

Governance & Security Implementation Summary

Component	Purpose	Business Benefit
Workspace Access	Manage user permissions	Secure collaboration and controlled access
Pipeline Monitoring	Track orchestration activities	Detect failures and monitor ETL execution
Dataflow Monitoring	Monitor data transformation jobs	Ensure successful refresh operations
Semantic Model Connection	Connect Power BI to Fabric	Centralized business logic and reusable metrics
Dashboard Development	Build interactive business reports	Enable data-driven decision making
Report Publishing	Publish reports to Fabric	Secure sharing and centralized access
Report Validation	Verify report functionality	Ensure accurate reporting and reliable analytics

Module 5: Deployment Pipeline

Overview

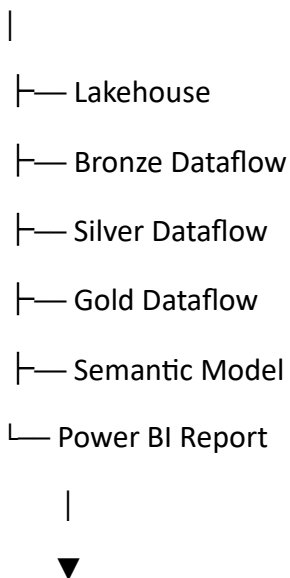
Deployment Pipelines in Microsoft Fabric enable organizations to promote analytics solutions across multiple environments while maintaining consistency and governance. Instead of rebuilding artifacts for each environment, Deployment Pipelines allow validated artifacts to be promoted from **Development** to **Test**, and finally to **Production**.

For this project, the Development workspace contains the complete Banking Analytics solution. The Test workspace contains only the artifacts required for validation, while the Production workspace contains the final reporting assets that are consumed by business users.

The Deployment Pipeline implementation consists of:

- Create Development, Test, and Production Workspaces
- Create a Deployment Pipeline
- Assign Workspaces to Pipeline Stages
- Deploy Artifacts from Development to Test
- Validate the Test Environment
- Deploy Artifacts from Test to Production
- Validate the Production Environment

Development



Test

|

|— Gold Dataflow

|— Semantic Model

└─ Power BI Report

|



Production

|

|— Semantic Model

└─ Power BI Report



5.1 Create Environment Workspaces

OBJECTIVE

Create separate Microsoft Fabric workspaces for Development, Test, and Production environments to support controlled application lifecycle management.

NAVIGATION

Microsoft Fabric

→ Workspaces

→ New Workspace

METHOD

Follow the steps below in sequence:

1. Open Microsoft Fabric.
2. Click Workspaces.
3. Select New Workspace.
4. Create the Development workspace:

fabric_banking_analytics_ws

5. Create the Test workspace:

fabric_banking_analytics_test_ws

6. Create the Production workspace:

fabric_banking_analytics_prod_ws

7. Verify that all three workspaces appear in the Workspaces list.
-

EXPECTED RESULT

Three dedicated workspaces are successfully created for Development, Test, and Production.

5.2 Create the Deployment Pipeline

OBJECTIVE

Create a Deployment Pipeline to manage the promotion of Microsoft Fabric artifacts across multiple environments.

NAVIGATION

Microsoft Fabric

→ Deployment Pipelines

→ New Pipeline

METHOD

Follow the steps below in sequence:

1. Open Microsoft Fabric.
2. Select Deployment Pipelines.
3. Click New Pipeline.
4. Enter the following name:

Banking Analytics Deployment Pipeline

5. Click Create.
 6. Verify that the Deployment Pipeline opens successfully.
-

EXPECTED RESULT

A Deployment Pipeline is successfully created.

5.3 Assign Workspaces to Pipeline Stages

OBJECTIVE

Associate each Fabric workspace with the corresponding Deployment Pipeline stage.

NAVIGATION

Deployment Pipeline

→ Assign Workspace

METHOD

Follow the steps below in sequence:

Development Stage

1. Click Assign Workspace.
2. Select:

fabric_banking_analytics_ws

3. Click Assign.
-

Test Stage

4. Click Assign Workspace under Test.
5. Select:

fabric_banking_analytics_test_ws

6. Click Assign.
-

Production Stage

7. Click Assign Workspace under Production.
8. Select:

fabric_banking_analytics_prod_ws

9. Click Assign.

10. Verify that all three stages display the correct workspace names.

EXPECTED RESULT

Development, Test, and Production stages are successfully mapped to their respective Fabric workspaces.

5.4 Deploy Development to Test

OBJECTIVE

Promote validated analytical artifacts from the Development environment to the Test environment.

NAVIGATION

Deployment Pipeline

→ Development

→ Deploy

METHOD

Follow the steps below in sequence:

1. Open the Deployment Pipeline.
 2. Select the Development stage.
 3. Click Deploy.
 4. Review the deployment summary.
 5. Verify that the following artifacts are selected:
 - Gold Dataflow
 - Semantic Model
 - Power BI Report
 6. Click Deploy.
 7. Wait for deployment to complete.
 8. Verify that all artifacts are successfully promoted.
-

5.5 Validate the Test Environment

OBJECTIVE

Verify that the promoted artifacts function correctly within the Test environment before releasing them to Production.

NAVIGATION

fabric_banking_analytics_test_ws

METHOD

Follow the steps below in sequence:

1. Open the Test workspace.
2. Verify the following artifacts exist:
 - Gold Dataflow
 - Semantic Model
 - Power BI Report
3. Open the Semantic Model.
4. Confirm that all tables and relationships are available.
5. Open the Power BI Report.
6. Verify:
 - KPI Cards
 - Charts
 - Slicers
 - Filters
 - Drill-through
 - Bookmarks
 - Tooltips
7. Ensure that the report loads successfully without errors.

5.6 Deploy Test to Production

OBJECTIVE

Promote the approved analytical solution from the Test environment into the Production environment.

NAVIGATION

Deployment Pipeline

→ Test

→ Deploy

METHOD

Follow the steps below in sequence:

1. Return to the Deployment Pipeline.
 2. Select the Test stage.
 3. Click Deploy.
 4. Review the deployment summary.
 5. Verify that the following artifacts are selected:
 - Semantic Model
 - Power BI Report
 6. Click Deploy.
 7. Wait until deployment completes successfully.
-

EXPECTED RESULT

The approved Semantic Model and Power BI Report are successfully promoted to the Production workspace.

5.7 Validate the Production Environment

OBJECTIVE

Verify that the Production workspace contains the approved reporting assets and is ready for business users.

NAVIGATION

fabric_banking_analytics_prod_ws

METHOD

Follow the steps below in sequence:

1. Open the Production workspace.
 2. Verify the following artifacts exist:
 - Semantic Model
 - Power BI Report
 3. Open the report.
 4. Verify:
 - KPI Cards display correctly.
 - Charts load successfully.
 - Filters and slicers function correctly.
 - Report interactions perform as expected.
 5. Confirm that the report is connected to the Production Semantic Model.
-

EXPECTED RESULT

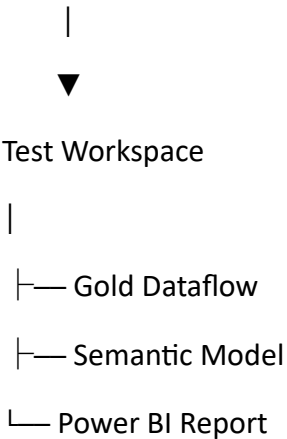
The Production workspace contains a fully functional reporting solution ready for business users.

Deployment Architecture

Development Workspace



Deployment Pipeline



Deployment Pipeline



Production Workspace

|

|— Semantic Model

└— Power BI Report

Deployment Pipeline Summary

Stage	Purpose	Artifacts
Development	Active development and implementation	Lakehouse, Dataflows, Semantic Model, Report
Test	Validation and quality assurance	Gold Dataflow, Semantic Model, Report
Production	Business consumption	Semantic Model, Report

Module 6: Materialized Lake Views

Overview

Materialized Lake Views (MLVs) in Microsoft Fabric provide a mechanism for precomputing and storing query results on Delta tables within a Lakehouse. Unlike standard SQL views, Materialized Lake Views physically persist aggregated results, significantly improving query performance for frequently executed analytical workloads.

In this implementation, Materialized Lake Views are created over the curated Gold-layer tables to accelerate reporting and business intelligence queries. The refresh process is automated through scheduled lineage runs, ensuring that the views remain synchronized with the latest Gold-layer data.

The Materialized Lake View implementation consists of:

- Create Materialized Lake Views
 - Validate Materialized Lake Views
 - View Extended Lineage
 - Configure Refresh Schedule
 - Execute Scheduled Refresh
 - Monitor Recent Runs
-

6.1 Create Materialized Lake Views

OBJECTIVE

Create Materialized Lake Views on the Gold-layer tables to precompute commonly used aggregations and improve query performance for analytical workloads.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ fabric_banking_medallion_lakehouse

→ Materialized Views

→ New Notebook (Spark SQL)

METHOD

Follow the steps below in sequence:

1. Open the **fabric_banking_analytics_ws** workspace.
2. Open the following Lakehouse:

fabric_banking_medallion_lakehouse

3. Select **Materialized Views**.
4. Click **New Notebook**.
5. Change the notebook language to:

Spark SQL

6. **Create the first Materialized Lake View.**

```
CREATE MATERIALIZED LAKE VIEW mlv_customer_summary
```

```
AS
```

```
SELECT
```

```
  gender,
```

```
COUNT(*) AS total_customers,  
AVG(credit_score) AS average_credit_score,  
AVG(income) AS average_income,  
AVG(debt) AS average_debt,  
SUM(number_of_cards) AS total_cards  
FROM dim_customer  
GROUP BY gender;
```

7. Execute the notebook cell.

8. Repeat the process to create additional Materialized Lake Views for card and transaction analytics.

Example:

- mlv_card_summary
- mlv_transaction_summary

9. Wait until all Materialized Lake Views are successfully created.

EXPECTED RESULT

Three Materialized Lake Views are successfully created and appear under the **Materialized Views** section of the Lakehouse.

6.2 Validate Materialized Lake Views

OBJECTIVE

Verify that the Materialized Lake Views have been created successfully and contain the expected precomputed analytical data.

NAVIGATION

fabric_banking_edallion_lakehouse

→ Materialized Views

METHOD

Follow the steps below in sequence:

1. Expand the **Materialized Views** folder.
 2. Verify the following views exist:
 - mlv_customer_summary
 - mlv_card_summary
 - mlv_transaction_summary
 3. Open each Materialized Lake View.
 4. Preview the generated data.
 5. Confirm that aggregated values appear correctly.
 6. Verify that the schema matches the expected output.
-

EXPECTED RESULT

All Materialized Lake Views are successfully created and display precomputed analytical results.

6.3 View Extended Lineage

OBJECTIVE

Visualize the upstream and downstream dependencies of Materialized Lake Views to understand how analytical data flows through the Medallion Architecture.

NAVIGATION

fabric_banking_medallion_lakehouse

→ Materialized Views

→ Lineage

METHOD

Follow the steps below in sequence:

1. Open the Lakehouse.
 2. Select **Materialized Views**.
 3. Open the Lineage view.
 4. Review the dependency graph.
 5. Verify that each Materialized Lake View consumes data from the Gold Delta tables.
 6. Confirm that lineage correctly displays the relationship between:
 - Gold Delta Tables
 - Materialized Lake Views
 7. Review the refresh dependency flow.
-

EXPECTED RESULT

The Lineage view accurately represents the dependency chain between the Gold-layer tables and the Materialized Lake Views.

6.4 Configure Scheduled Refresh

OBJECTIVE

Configure an automated refresh schedule to keep the Materialized Lake Views synchronized with changes in the Gold-layer data.

NAVIGATION

fabric_banking_medallion_lakehouse

→ Materialized Views

→ Schedule

METHOD

Follow the steps below in sequence:

1. Open the Materialized Views Manager.
2. Select **Schedule**.
3. Enable scheduled refresh.
4. Configure the refresh frequency.

Example:

Daily

5. Configure the execution time.
 6. Save the schedule.
 7. Verify that the schedule appears in the Materialized Views Manager.
-

EXPECTED RESULT

An automated refresh schedule is successfully configured for the Materialized Lake Views.

6.5 Execute the Scheduled Refresh

OBJECTIVE

Execute the configured refresh schedule to regenerate the Materialized Lake Views using the latest Gold-layer data.

NAVIGATION

fabric_banking_medallion_lakehouse

→ Materialized Views

→ Run Schedule

METHOD

Follow the steps below in sequence:

1. Open the Materialized Views Manager.
 2. Select the configured schedule.
 3. Click **Run Now**.
 4. Monitor the execution.
 5. Wait for the refresh process to complete.
 6. Verify that all Materialized Lake Views refresh successfully.
-

EXPECTED RESULT

The scheduled refresh executes successfully and updates all Materialized Lake Views.

6.6 Monitor Recent Runs

OBJECTIVE

Monitor recent Materialized Lake View refresh executions and verify successful completion of scheduled lineage runs.

NAVIGATION

fabric_banking_medallion_lakehouse

→ Materialized Views

→ Recent Runs

METHOD

Follow the steps below in sequence:

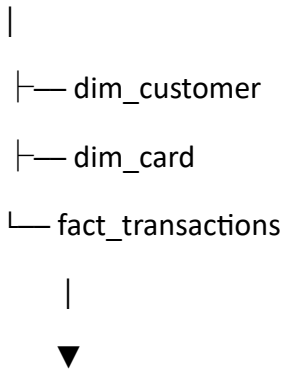
1. Open the Materialized Views Manager.
 2. Select **Recent Runs**.
 3. Review the refresh history.
 4. Verify the following information:
 - Run Status
 - Start Time
 - End Time
 - Duration
 - Refresh Type
 5. Confirm that the latest refresh completed successfully.
 6. Review any previous execution history if available.
-

EXPECTED RESULT

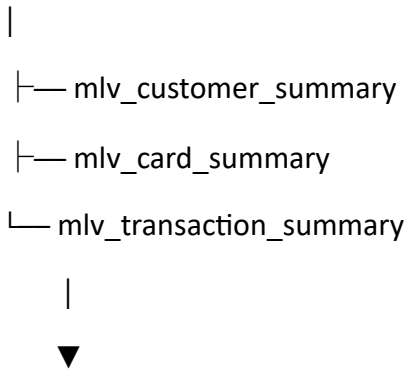
The Recent Runs page displays successful execution history for all scheduled Materialized Lake View refreshes.

Materialized Lake View Architecture:

Gold Delta Tables



Materialized Lake Views



Precomputed Analytical Results



Power BI Reporting

Materialized Lake View Summary

Materialized Lake View	Source Table	Purpose
mlv_customer_summary	dim_customer	Customer-level business aggregations
mlv_card_summary	dim_card	Card-related analytical summaries
mlv_transaction_summary	fact_transactions	Transaction-level aggregated metrics

Module 7: Git Integration

Overview

Microsoft Fabric Git Integration enables version control, collaboration, and lifecycle management by connecting Fabric workspaces with Git repositories. Instead of maintaining artifacts only within a workspace, Git Integration stores supported Fabric artifacts in a GitHub repository, allowing multiple developers to collaborate using modern DevOps practices.

In this implementation, the complete Banking Analytics workspace is synchronized with GitHub, including Dataflows, Pipelines, Lakehouse, Notebooks, Semantic Models, and Power BI Reports. A second Microsoft Fabric workspace is then connected to the same repository to demonstrate collaborative development.

The Git Integration implementation consists of:

- Configure GitHub Prerequisites
 - Generate a GitHub Personal Access Token
 - Create a GitHub Repository
 - Connect the Fabric Workspace to GitHub
 - Synchronize Workspace Artifacts
 - Verify Repository Contents
 - Synchronize Incremental Changes
 - Configure a Second Developer
 - Synchronize a Second Fabric Workspace
-

7.1 Configure GitHub Prerequisites

OBJECTIVE

Prepare the GitHub environment required for Microsoft Fabric Git Integration by creating a repository and granting repository access to collaborators.

NAVIGATION

GitHub

→ New Repository

METHOD

Follow the steps below in sequence:

1. Sign in to your GitHub account.
2. Click **New Repository**.
3. Configure the repository using the following settings:

Repository Name

fabric-banking-analytics

Visibility

Private

4. Click **Create Repository**.
5. If collaborative development is required, open:

Repository

→ Settings

→ Collaborators

6. Click **Add people**.
 7. Invite the second developer.
 8. Wait for the collaborator to accept the invitation.
-

7.2 Generate a GitHub Personal Access Token

OBJECTIVE

Generate a GitHub Personal Access Token (PAT) that allows Microsoft Fabric to securely authenticate with the GitHub repository.

NAVIGATION

GitHub

→ Profile

→ Settings

→ Developer Settings

→ Personal Access Tokens

→ Fine-grained Tokens

METHOD

Follow the steps below in sequence:

1. Open GitHub Settings.

2. Navigate to:

Developer Settings

→ Personal Access Tokens

3. Click **Generate New Token**.

4. Enter a descriptive name.

Example:

Fabric Git Integration

5. Select the repository:

fabric-banking-analytics

6. Grant the required repository permissions.

7. Generate the token.
8. Copy the generated token immediately.
9. Store the token securely.

EXPECTED RESULT

A GitHub Personal Access Token is generated and ready for authentication.

7.3 Connect the Fabric Workspace to GitHub

OBJECTIVE

Connect the Development workspace to the GitHub repository to enable source control and version management.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_ws

→ Workspace Settings

→ Git Integration

METHOD

Follow the steps below in sequence:

1. Open the **fabric_banking_analytics_ws** workspace.
2. Select:

Workspace Settings

3. Open the **Git Integration** tab.
4. Click **Connect to Git**.
5. Configure:

Provider

GitHub

Repository

fabric-banking-analytics

Branch

main

6. Authenticate using the GitHub Personal Access Token generated from **your own GitHub account**.

7. Complete the connection.
8. Wait until the workspace displays:

Connected

EXPECTED RESULT

The Development workspace is successfully connected to the GitHub repository.

7.4 Synchronize Workspace Artifacts

OBJECTIVE

Synchronize supported Microsoft Fabric artifacts from the Development workspace into the connected GitHub repository.

NAVIGATION

fabric_banking_analytics_ws

→ Source Control

METHOD

Follow the steps below in sequence:

1. Open the Source Control pane.
2. Verify that the workspace is connected to GitHub.
3. Review the synchronization status of the workspace artifacts.
4. If the repository is empty, allow Microsoft Fabric to automatically initialize the repository during the first synchronization.
5. Wait until synchronization completes.
6. Confirm that all supported artifacts display the status:

Synced

EXPECTED RESULT

All supported Fabric artifacts are successfully synchronized with the GitHub repository.

7.5 Verify Repository Structure

OBJECTIVE

Verify that Microsoft Fabric has successfully synchronized workspace artifacts into the GitHub repository.

NAVIGATION

GitHub

→ fabric-banking-analytics

METHOD

Follow the steps below in sequence:

1. Open the GitHub repository.
 2. Refresh the repository.
 3. Review the synchronized folder structure.
 4. Verify that the repository contains Fabric artifacts such as:
 - Lakehouse
 - Dataflows
 - Pipelines
 - Notebooks
 - Semantic Models
 - Power BI Reports
 5. Review the commit history.
 6. Verify that the synchronization completed successfully.
-

EXPECTED RESULT

The GitHub repository contains the synchronized Microsoft Fabric project.

7.6 Synchronize Workspace Changes

OBJECTIVE

Synchronize modifications made to Fabric artifacts with the connected GitHub repository.

NAVIGATION

fabric_banking_analytics_ws

→ Source Control

METHOD

Follow the steps below in sequence:

1. Modify an existing Fabric artifact.

Example:

- Dataflow
 - Pipeline
 - Notebook
 - Semantic Model
2. Save the changes.
 3. Return to the Source Control pane.
 4. Observe that the modified artifact displays a pending synchronization status.
 5. Synchronize the workspace.
 6. Wait until synchronization completes successfully.
 7. Open GitHub.
 8. Verify that the repository reflects the updated artifact.
-

EXPECTED RESULT

Modified workspace artifacts are successfully synchronized with GitHub.

7.7 Configure a Second Developer

OBJECTIVE

Prepare a second developer for collaborative Microsoft Fabric development using the shared GitHub repository.

NAVIGATION

GitHub

→ Repository

→ Settings

→ Collaborators

METHOD

Follow the steps below in sequence:

1. Open the GitHub repository.
2. Navigate to:

Settings

→ Collaborators

3. Add the second developer.
 4. Wait for the invitation to be accepted.
 5. Ask the second developer to generate **their own** GitHub Personal Access Token from **their own GitHub account**.
 6. Confirm that the second developer has access to the shared repository.
-

EXPECTED RESULT

The second developer is granted secure access to the GitHub repository.

7.8 Synchronize a Second Fabric Workspace

OBJECTIVE

Synchronize a second Microsoft Fabric workspace with the shared GitHub repository, allowing another developer to retrieve the latest Banking Analytics solution.

NAVIGATION

Microsoft Fabric

→ fabric_banking_analytics_dev_ws

→ Workspace Settings

→ Git Integration

METHOD

Follow the steps below in sequence:

1. Sign in using the second Microsoft Fabric account.

2. Open:

fabric_banking_analytics_dev_ws

3. Navigate to:

Workspace Settings

→ Git Integration

4. Click **Connect to Git**.

5. Configure:

Provider

GitHub

Repository

fabric-banking-analytics

Branch

main

6. Authenticate using the **GitHub Personal Access Token** generated from the **second developer's GitHub account**.
7. Complete the Git connection.
8. Click:

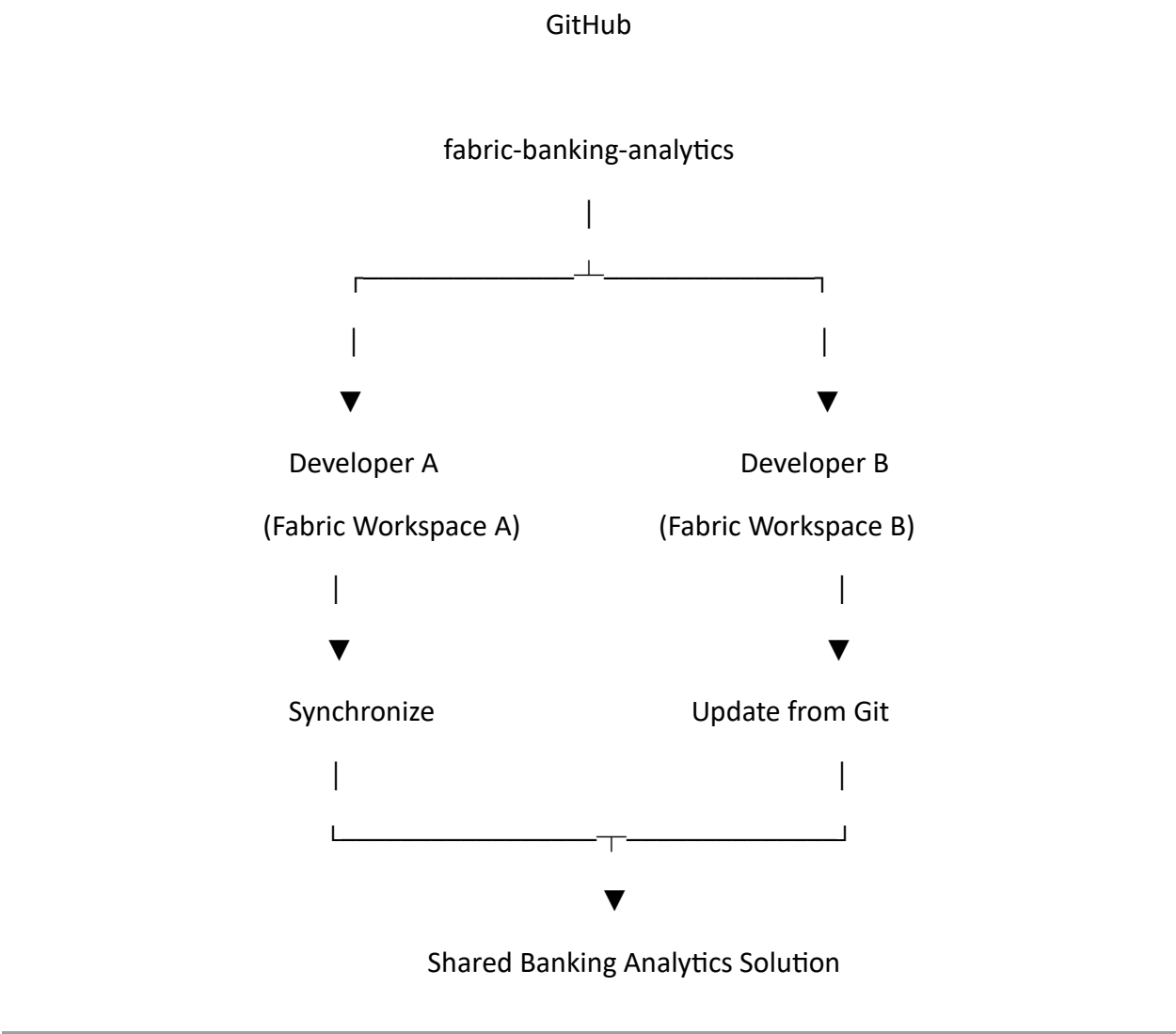
Update From Git

9. Wait until synchronization completes.
10. Verify that the Fabric artifacts appear in the Development workspace.

EXPECTED RESULT

The second Microsoft Fabric workspace successfully retrieves the latest version of the Banking Analytics solution from the shared GitHub repository.

Git Integration Architecture



Git Integration Summary

Component	Purpose	Business Benefit
GitHub Repository	Central source control	Stores Fabric artifacts under version control
Personal Access Token	Secure authentication	Enables Fabric to connect to GitHub securely
Workspace Synchronization	Publish workspace changes	Maintains a synchronized project repository
Source Control	Track modifications	Provides version history and change tracking
Collaboration	Multiple developers	Supports parallel development using a shared repository
Update From Git	Retrieve latest changes	Keeps workspaces synchronized with the repository

Benefits of Git Integration

- Enables enterprise-grade source control for Microsoft Fabric artifacts.
 - Supports collaborative development across multiple developers.
 - Maintains a complete version history of workspace changes.
 - Simplifies rollback to previous versions when required.
 - Synchronizes Lakehouses, Dataflows, Pipelines, Notebooks, Semantic Models, and Reports.
 - Integrates Microsoft Fabric with modern DevOps workflows.
 - Provides a centralized repository for managing analytics solutions.
-

Conclusion

The **Microsoft Fabric Banking Analytics Project** demonstrates the implementation of a complete end-to-end modern analytics solution using the unified capabilities of Microsoft Fabric. By following the **Medallion Architecture**, the project successfully transforms raw banking data into trusted, business-ready analytical assets that support enterprise reporting and decision-making.

The implementation begins with the **Bronze layer**, where raw banking datasets are ingested and stored in their original format using Microsoft Fabric Pipelines, Dataflow Gen2, and Delta Tables. The **Silver layer** enhances data quality by applying cleansing, standardization, validation, and data enrichment techniques, resulting in consistent and reliable datasets. Building upon this foundation, the **Gold layer** creates business-oriented Dimension and Fact tables, establishes a Star Schema, and develops a centralized Semantic Model with reusable DAX measures to support interactive analytics and self-service reporting.

Beyond data engineering, the project incorporates key enterprise capabilities including **Governance & Security**, where workspace permissions, pipeline execution monitoring, dataflow refresh monitoring, and secure report publishing ensure a controlled and well-governed analytics environment. The implementation further demonstrates a structured **Deployment Pipeline**, enabling the controlled promotion of artifacts across Development, Test, and Production environments while supporting an enterprise-grade application lifecycle.

To improve analytical performance, **Materialized Lake Views** are implemented over the Gold-layer tables, providing precomputed aggregations, scheduled refreshes, lineage visibility, and faster query execution for reporting workloads. Finally, **Git Integration** extends the solution into a collaborative development environment by connecting Microsoft Fabric with GitHub, enabling version control, source management, workspace synchronization, and multi-developer collaboration through modern DevOps practices.

Collectively, these implementations showcase how Microsoft Fabric unifies **data ingestion, engineering, storage, modeling, governance, reporting, optimization, deployment, and collaboration** within a single platform. The Banking Analytics solution not only demonstrates the practical application of Microsoft Fabric services but also follows industry-recognized architectural patterns and enterprise best practices for building scalable, maintainable, and production-ready analytics solutions.

This implementation guide serves as a comprehensive reference for developers, data engineers, business intelligence professionals, and learners seeking to understand and implement an end-to-end analytics solution using Microsoft Fabric. By following the documented steps and methodologies, organizations and individuals can reproduce the complete Banking Analytics

solution, adapt it to their own business scenarios, and leverage Microsoft Fabric to build secure, performant, and collaborative data platforms for modern enterprise analytics.
